

ISC 4242

Project Report

Car Price Prediction Using Machine Learning

Ivan Rey Pestana

UCF: 3706230

ISC 4242

Teng Zhang

Spring 2026

Table of Contents

1. Introduction	3
2. Dataset Description	3
3. Data Selection and Preparation	4
3.1 Data Cleaning	4
3.2 Feature Engineering and Preprocessing	4
3.3 Train-Test Split	5
4. Exploratory Data Analysis	5
4.1 Price Distribution	5
4.2 Horsepower vs. Price	6
4.3 Correlation Heatmap	6
5. Machine Learning Models	7
5.1 Linear Regression	7
5.2 K-Nearest Neighbors (KNN) Regression	7
5.3 Support Vector Machine (SVM) Regression	8
6. Model Comparison and Results	8
7. Conclusion	9
References	10
Appendix: Full Project Code	11

1. Introduction

This project is about predicting car prices using machine learning models. The goal is to understand how different car features like horsepower, engine size, and speed affect the price of a vehicle. Accurately estimating a vehicle's price is a relevant and practical problem that benefits both consumers and dealers. By analyzing key automotive features such as horsepower, engine size, top speed, and fuel type, the goal is to build models that can predict the price of a car based on its specifications.

The dataset used is CarsDatasets2025.csv, which contains information about a wide variety of vehicles from different manufacturers. This project follows a typical data science process: loading and cleaning the data, exploring it, preparing it for modeling, training models, and then comparing their performance.

Three regression algorithms are tested and compared: Linear Regression, K-Nearest Neighbors (KNN) Regression, and Support Vector Machine (SVM) Regression. Performance is measured using Root Mean Square Error (RMSE) and the R-squared (R2) coefficient.

2. Dataset Description

The dataset CarsDatasets2025.csv contains records for a variety of car models across multiple manufacturers. Each row represents a unique car, and the dataset contains the following 11 columns:

Column Name	Description	Type
Company Names	The manufacturer or brand of the car (e.g., BMW, Toyota)	Categorical
Cars Names	The specific model name of the vehicle	Categorical
Engines	Engine type or configuration description	Categorical
CC/Battery Capacity	Engine displacement in cubic centimeters, or battery capacity in mAh	Numerical
HorsePower	Engine power output measured in horsepower (HP)	Numerical
Total Speed	Maximum top speed in km/h	Numerical
Performance (0-100 KM/H)	Acceleration time from 0 to 100 km/h in seconds	Numerical
Cars Prices	Price of the car in USD (target variable)	Numerical
Fuel Types	Type of fuel used (e.g., gasoline, electric, diesel)	Categorical

Seats	Number of passenger seats	Numerical
Torque	Engine torque measured in Newton-meters (Nm)	Numerical

The target variable for prediction is Cars Prices. The remaining columns serve as features. Four columns are categorical (Company Names, Cars Names, Engines, Fuel Types) and six are numerical. The dataset required cleaning before being using it, as many fields contained measurement units (e.g., "963 hp", "\$1,100,000"), ranges (e.g., "100-120"), or special characters.

3. Data Selection and Preparation

3.1 Data Cleaning

Before using the dataset, cleaning was necessary because many values were not in a usable format. First, all rows containing missing values (NaN) were dropped using the `dropna()` method. This ensures that no incomplete records are passed to the models.

Next, a custom cleaning function was applied to all numerical columns. This function removed common unit suffixes such as "\$", ",", "hp", "km/h", "sec", "nm", "cc", and "mah" from the values. For values that contained a slash ("/"), only the portion before the slash was kept. For values that represented a range using a hyphen (e.g., "100-120"), the average of the two endpoints was computed. After removing all non-numeric characters, the values were converted to floating-point numbers. If conversion failed, the value was replaced with NaN.

A separate cleaning function was applied to the Seats column to handle values like "2+2", where the total was computed as the sum of the parts. The cleaned numerical columns were: HorsePower, Total Speed, Performance (0-100 KM/H), Torque, CC/Battery Capacity, and Seats. Cars Prices was also cleaned using the same numeric function.

3.2 Feature Engineering and Preprocessing

After cleaning the data, it was separated it into features (X) and the target variable (y), which is the car price. The features were divided into two groups:

- Numerical columns: HorsePower, Total Speed, Performance (0-100 KM/H), Torque, CC/Battery Capacity, Seats
- Categorical columns: Company Names, Cars Names, Engines, Fuel Types

A `ColumnTransformer` was used to apply two different preprocessing strategies simultaneously. `StandardScaler` was applied to the numerical columns to normalize them to have a mean of 0 and a standard deviation of 1. This is important for algorithms such as KNN and SVM, which are sensitive to feature scale. `OneHotEncoder` was applied to the categorical columns, with `handle_unknown="ignore"` to avoid errors on unseen categories in the test set. This converts each unique category into a binary column, allowing the models to work with non-numeric data.

3.3 Train-Test Split

The preprocessed data was split into training and testing sets using an 80/20 ratio. 80% of the records were used to train the models, and the remaining 20% were held out for evaluation. A `random_state` of 42 was used to ensure reproducibility of the split.

4. Exploratory Data Analysis

Before training the models, I explored the data to better understand the distribution of the data and the relationships between the features and the target variable.

4.1 Price Distribution

The histogram below shows the distribution of car prices in the dataset. The distribution is heavily right-skewed, meaning that most cars fall within the lower price range, while a small number of high-end vehicles have significantly higher prices. This can be considered typical since luxury and exotic vehicles represent outliers. The skewness of the price distribution can make it challenging for linear models to make accurate predictions without additional transformations.

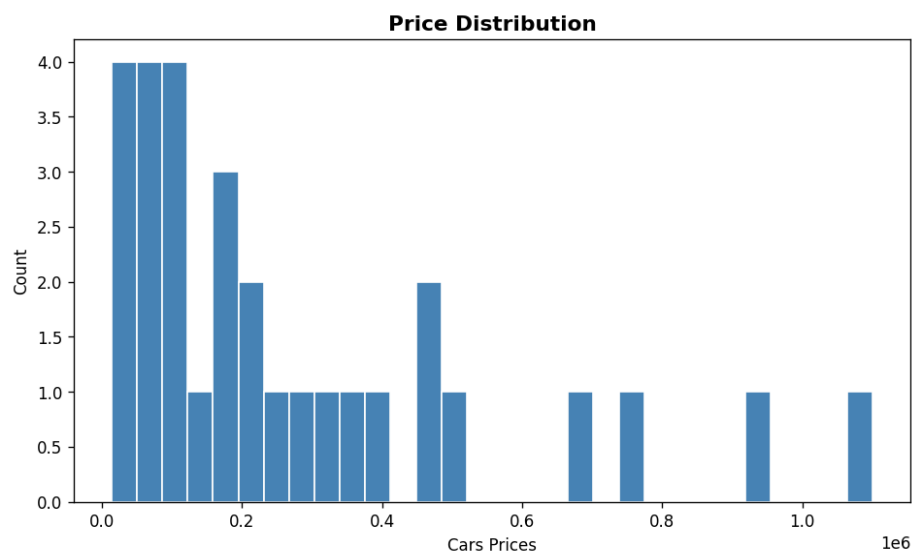


Figure 1. Distribution of Car Prices in the Dataset

4.2 HorsePower vs. Price

The scatter plot below illustrates the relationship between a car's horsepower and its price. Cars with higher horsepower tend to have higher prices. This makes sense, as performance-oriented vehicles typically command a premium in the market. The correlation is not perfectly linear, and some variability is observed, suggesting that other features also contribute significantly to price. A few high-horsepower, extremely expensive vehicles appear as outliers in the upper-right region of the chart.

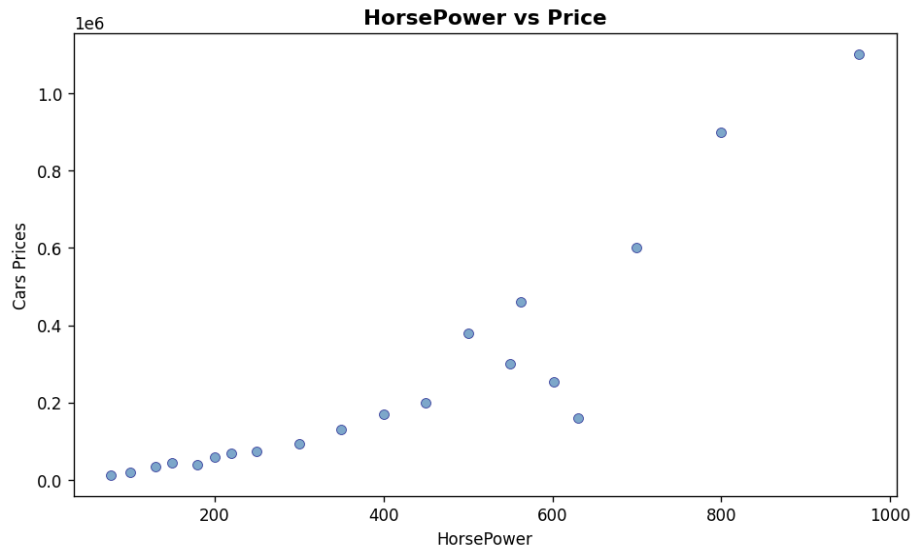


Figure 2. HorsePower vs. Cars Prices Scatter Plot

4.3 Correlation Heatmap

The correlation heatmap below shows the pairwise Pearson correlation coefficients between all numerical features and the target variable (Cars Prices). Key observations include:

- HorsePower has the strongest positive correlation with Cars Prices (~ 0.74), making it the most predictive single feature.
- Total Speed also shows a strong positive correlation (~ 0.66), indicating faster cars tend to be more expensive.
- Performance (0-100 KM/H) shows a negative correlation (~ -0.55), as lower acceleration times (i.e., faster cars) correspond to higher prices.
- Number of Seats shows little correlation with price, suggesting that seating capacity alone does not drive vehicle cost.

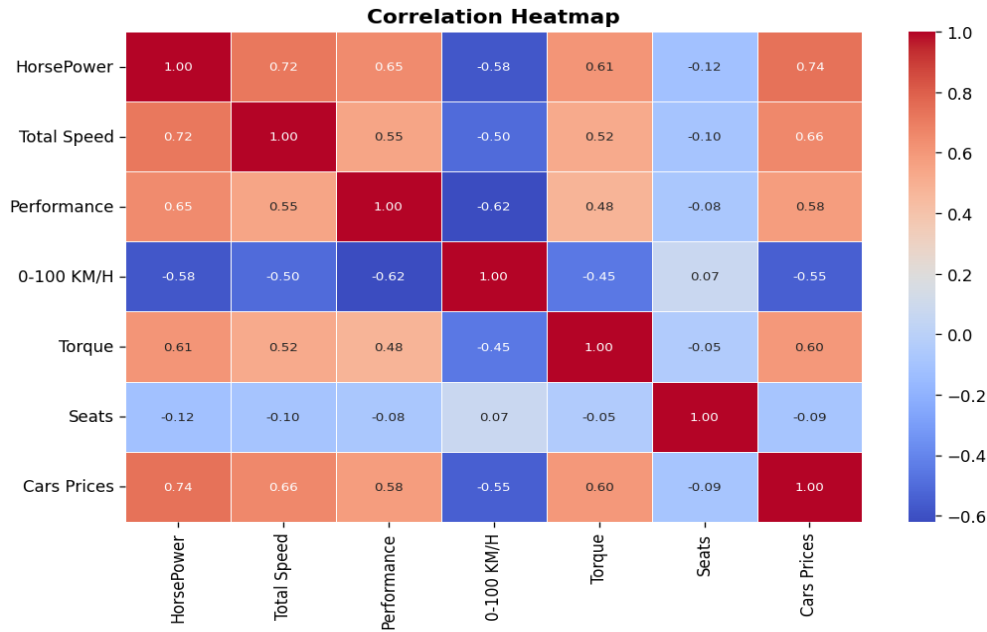


Figure 3. Correlation Heatmap of Numerical Features

5. Machine Learning Models

Three regression models were trained on the preprocessed data to predict car prices. Each model was evaluated using two metrics: Root Mean Square Error (RMSE), which measures the average magnitude of prediction error in the same units as the target variable (USD), and R-squared (R2), which indicates how well the model explains the variance in the target variable.

5.1 Linear Regression

Linear Regression is a simple model that assumes a straight-line relationship between the features and the price. It assumes a linear relationship between the input features and the target variable, learning a set of weights (coefficients) for each feature.

The model was trained using Scikit-learn's LinearRegression class with default parameters. Results on the test set:

Metric	Value	Interpretation
RMSE	128,529.04	Average prediction error of ~\$128,529 per car
R2 Score	-0.7284	Model performs worse than a simple mean predictor

The negative R2 score indicates that the Linear Regression model is a poor fit for this dataset. This is likely due to the non-linear and complex relationships between car specifications and prices,

which a simple linear model cannot capture effectively. The high RMSE also confirms that the predictions deviate significantly from the actual prices on average.

5.2 K-Nearest Neighbors (KNN) Regression

KNN works differently — it looks at similar data points and averages their values. Rather than learning a global model, KNN predicts a new value by finding the K most similar (nearest) training examples and averaging their target values. Similarity is typically measured using Euclidean distance in feature space.

The model was configured with `n_neighbors=5`, meaning it considers the 5 nearest neighbors when making each prediction. Results:

Metric	Value	Interpretation
RMSE	95,847.78	Average prediction error of ~\$95,848 per car
R2 Score	0.0388	Model explains ~3.9% of the variance in price

KNN produced the best results of the three models, with the lowest RMSE and the only positive R2 score. While an R2 of 0.039 is still very low, it means the model does slightly better than predicting the mean price for every car. The improvement over Linear Regression suggests that the price-feature relationship has some local structure that KNN can partially capture. A larger value of K or feature engineering could potentially improve performance.

5.3 Support Vector Machine (SVM) Regression

SVM tries to model non-linear relationships using kernels. fits a hyperplane in a high-dimensional space that best approximates the data points within a defined margin (epsilon tube). It uses kernel functions to handle non-linear relationships.

The model was configured with a radial basis function (RBF) kernel and a regularization parameter `C=100`. Results:

Metric	Value	Interpretation
RMSE	100,168.36	Average prediction error of ~\$100,168 per car
R2 Score	-0.0498	Marginally worse than a simple mean predictor

The SVM model performed slightly worse than KNN but significantly better than Linear Regression in terms of RMSE. The negative R2 score suggests the model still struggles to explain the variance in car prices. This may be because the RBF kernel and the current hyperparameter

settings ($C=100$) are not optimally tuned for this dataset. With proper hyperparameter optimization (e.g., grid search for C , epsilon, and gamma), SVM could potentially perform better.

6. Model Comparison and Results

The following table and chart summarize the performance of all three models on the test set. The best-performing model for this dataset, based on both RMSE and R^2 score, is KNN Regression.

Model	RMSE (\$)	R2 Score	Rank
K-Nearest Neighbors (k=5)	95,847.78	0.039	1st (Best)
SVM (RBF, C=100)	100,168.36	-0.050	2nd
Linear Regression	128,529.04	-0.728	3rd (Worst)

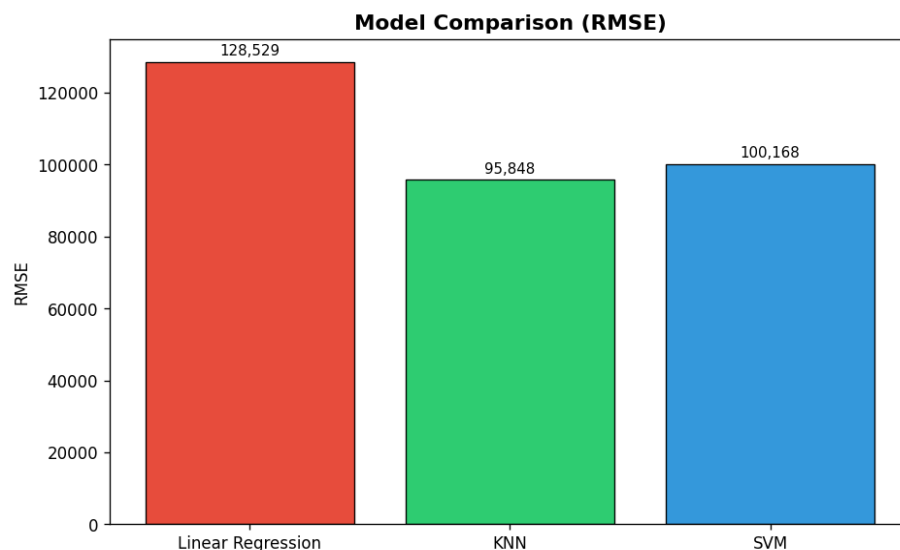


Figure 4. Bar Chart Comparing RMSE Across All Three Models

As visible in the bar chart, KNN has the shortest bar (lowest RMSE), confirming it as the best predictor among the three models tested. Linear Regression has the tallest bar and the most negative R^2 score, indicating it is the least suitable model for this particular dataset. SVM falls in between, performing moderately.

7. Conclusion

In this project, I built and compared three machine learning models to predict car prices. based on vehicle specifications. Three regression models were built and evaluated on the CarsDatasets2025 dataset after thorough data cleaning, feature preprocessing, and exploratory analysis.

Overall, all three models produced relatively high prediction errors, which reflects the inherent difficulty of this prediction task. Car prices are influenced by many factors beyond raw specifications, and the dataset's price distribution is highly skewed, with a few very expensive exotic vehicles making it harder for models to generalize.

Key takeaways from this project include:

1. KNN Regression ($k=5$) achieved the best performance with an RMSE of ~\$95,848 and the only positive R^2 score (0.039), suggesting it captures some local patterns in the data.
2. Linear Regression was the worst-performing model, with a highly negative R^2 of -0.728, confirming that car price prediction requires models capable of capturing non-linear relationships.
3. SVM Regression fell in the middle, but its negative R^2 suggests it also needs further tuning to be effective.
4. HorsePower was identified as the single feature most correlated with price, while Seats showed little predictive value.

Future improvements could include applying a log transformation to the highly skewed price variable, tuning hyperparameters using cross-validation and grid search, experimenting with more powerful ensemble models such as Random Forest or Gradient Boosting, and engineering additional features such as price-per-horsepower ratios. These steps would likely lead to substantially improved prediction accuracy.

References

- Mustafa, A. (2025). *Cars Dataset 2025* [Data set]. Kaggle.
<https://www.kaggle.com/datasets/abdullahmustafa93/cars-dataset-2025>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830. <https://scikit-learn.org>
- The Pandas Development Team. (2024). *pandas-dev/pandas: Pandas (Version 2.x)* [Software]. Zenodo. <https://pandas.pydata.org>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90-95. <https://matplotlib.org>

Appendix: Full Project Code

The following code represents the complete Python implementation used in this project, organized by section as it appears in the Jupyter Notebook (project.ipynb).

Section 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
```

Section 2: Load Data

```
# Load the dataset
df = pd.read_csv("CarsDatasets2025.csv", encoding="latin-1")
df.head()

# Print column names
print(df.columns)

# Strip whitespace from column names
df.columns = df.columns.str.strip()
```

Section 3: Clean Numerical Columns

```
# Drop rows with any missing values
df = df.dropna()

# Function to clean numerical values with unit suffixes and ranges
def clean_number(value):
    value = str(value).lower()
    value = value.replace("$", "").replace(",", "").replace("hp", "")
    value = value.replace("km/h", "").replace("sec", "").replace("nm", "")
    value = value.replace("cc", "").replace("mah", "").strip()

    if "/" in value:
        value = value.split("/")[0].strip()

    if "-" in value:
        parts = value.split("-")
        try:
            return (float(parts[0]) + float(parts[1])) / 2
        except:
            return np.nan

    try:
        return float(value)
```

```

    except:
        return np.nan

# Function to clean the Seats column (handles values like "2+2")
def clean_seats(value):
    value = str(value).lower().strip()
    if "+" in value:
        parts = value.split("+")
        try:
            return sum([float(p.strip()) for p in parts])
        except:
            return np.nan
    try:
        return float(value)
    except:
        return np.nan

# Apply cleaning functions to all numerical columns
df["Cars Prices"] = df["Cars Prices"].apply(clean_number)
df["HorsePower"] = df["HorsePower"].apply(clean_number)
df["Total Speed"] = df["Total Speed"].apply(clean_number)
df["Performance(0 - 100 )KM/H"] = df["Performance(0 - 100
)KM/H"].apply(clean_number)
df["Torque"] = df["Torque"].apply(clean_number)
df["CC/Battery Capacity"] = df["CC/Battery Capacity"].apply(clean_number)
df["Seats"] = df["Seats"].apply(clean_seats)

```

Section 4: Feature Engineering and Train-Test Split

```

# Define feature matrix X and target variable y
X = df.drop("Cars Prices", axis=1)
y = df["Cars Prices"]

# Define categorical and numerical column groups
categorical_cols = ["Company Names", "Cars Names", "Engines", "Fuel Types"]
numerical_cols = ["HorsePower", "Total Speed", "Performance(0 - 100 )KM/H",
                  "Torque", "CC/Battery Capacity", "Seats"]

# Build preprocessing pipeline:
# - StandardScaler for numerical columns
# - OneHotEncoder for categorical columns
preprocessor = ColumnTransformer([
    ("num", StandardScaler(), numerical_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categorical_cols)
])

# Apply preprocessing
X_processed = preprocessor.fit_transform(X)

# Split into training (80%) and testing (20%) sets
X_train, X_test, y_train, y_test = train_test_split(
    X_processed, y, test_size=0.2, random_state=42
)

```

Section 5: Exploratory Data Analysis (EDA)

```

# --- Price Distribution ---
sns.histplot(y, bins=30)

```

```
plt.title("Price Distribution")
plt.show()

# --- HorsePower vs Price ---
sns.scatterplot(x=df["HorsePower"], y=df["Cars Prices"])
plt.title("HorsePower vs Price")
plt.show()

# --- Correlation Heatmap ---
sns.heatmap(df[numerical_cols + ["Cars Prices"]].corr(), annot=True)
plt.title("Correlation Heatmap")
plt.show()
```

Section 6: Linear Regression Model

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Train the model
lr = LinearRegression()
lr.fit(X_train, y_train)

# Predict on test set
y_pred_lr = lr.predict(X_test)

# Evaluate performance
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
r2_lr = r2_score(y_test, y_pred_lr)

print("Linear Regression RMSE:", rmse_lr)
print("Linear Regression R2:", r2_lr)
```

Section 7: KNN Regression Model

```
from sklearn.neighbors import KNeighborsRegressor

# Train the model with k=5 neighbors
knn = KNeighborsRegressor(n_neighbors=5)
knn.fit(X_train, y_train)

# Predict on test set
y_pred_knn = knn.predict(X_test)

# Evaluate performance
rmse_knn = np.sqrt(mean_squared_error(y_test, y_pred_knn))
r2_knn = r2_score(y_test, y_pred_knn)

print("KNN RMSE:", rmse_knn)
print("KNN R2:", r2_knn)
```

Section 8: SVM Regression Model

```
from sklearn.svm import SVR

# Train the model with RBF kernel and C=100
```

```
svm = SVR(kernel="rbf", C=100)
svm.fit(X_train, y_train)

# Predict on test set
y_pred_svm = svm.predict(X_test)

# Evaluate performance
rmse_svm = np.sqrt(mean_squared_error(y_test, y_pred_svm))
r2_svm = r2_score(y_test, y_pred_svm)

print("SVM RMSE:", rmse_svm)
print("SVM R2:", r2_svm)
```

Section 9: Model Comparison Bar Chart

```
# Compare all models by RMSE
models = ["Linear Regression", "KNN", "SVM"]
rmse_values = [rmse_lr, rmse_knn, rmse_svm]

plt.bar(models, rmse_values)
plt.title("Model Comparison (RMSE)")
plt.show()
```